



Práctica 01: Fundamentos **procesador de consultas** **interactivo**

Alumno: Francisco Gabriel Ruiz Ruiz

Asignatura: Bases de datos

Curso 2025-2026

Fecha de práctica: 10/10/2025



Índice

Índice	2
1. Introducción	4
2. Gestión de sesión	5
2.1. CONNECT	6
2.2. PASSWORD	7
2.3. DISCONNECT	8
2.4. EXIT	9
3. Gestión buffer de instrucciones	10
3.1. LIST	11
3.2. INPUT	13
3.3. APPEND	14
3.4. CHANGE	15
3.5. DEL	16
3.6. SAVE	18
3.7. GET	19
3.8. RUN	20
3.9. /	21
4. Gestión de scripts	22
4.1. EDIT	23
4.2. REMARK	25
4.3. START / @	26
5. Gestión del entorno SQL*Plus	27
5.1. DESCRIBE	28
5.2. SPOOL	29
5.3. SHOW	30
5.4. SET	32
5.5. HELP	33
5.6. HOST	34



6. Gestión de variables	35
Diferencia entre & y &&	36
6.1. DEFINE	37
6.2. UNDEFINE	38
6.3. ACCEPT	39
6.4. VARIABLE	41
6.5. PRINT	43
7. Presentación de resultados	45
7.1. PAUSE	46
7.2. PROMPT	47
7.3. TTITLE	48
7.4. BTITLE	50
Resultados de TTITLE y BTITLE	51
7.5. COLUMN	52
8. Referencias	53



1. Introducción

En esta práctica se estudia el procesador de consultas interactivo SQL*Plus, una herramienta incluida con el sistema gestor Oracle Database 12c. SQL*Plus permite conectarse a la base de datos, ejecutar sentencias **SQL** y **PL/SQL**, y administrar tanto sesiones como scripts desde la línea de comandos.

El objetivo de esta práctica es familiarizarse con las principales sentencias de control del entorno (como CONNECT o SPOOL), comprender su propósito y aprender a utilizarlas correctamente.

A continuación, se presenta un resumen de los comandos más relevantes, incluyendo su sintaxis, uso y ejemplo.



2. Gestión de sesión

Los comandos de gestión de sesión permiten **conectarse**, **desconectarse** y **cerrar SQL*Plus**, así como cambiar contraseñas. Una sesión es el vínculo activo entre el cliente SQL*Plus y la base de datos Oracle.



2.1. CONNECT

Establece conexión con una base de datos Oracle. Si el usuario ya está conectado, la nueva conexión reemplaza a la anterior.

Sintaxis

```
SQL
CONNECT usuario[/contraseña][@cadena_conexión]
```

Parámetros

- **usuario**: Nombre del usuario de la base de datos que inicia sesión.
- **contraseña**: Contraseña asociada al usuario (se usa el símbolo '/' como separador).
- **cadena_conexión**: es el identificador del servicio de la base de datos (si se omite, se usa la local). Se usa el símbolo '@' como separador.

Ejemplos

```
SQL
SQL> CONNECT
Enter user-name: alu0101618586
Enter password:
Connected.
```



SQL

```
SQL> CONNECT alu0101618586@ORCLPDB1
Enter password:
Connected.
```

2.2. PASSWORD

Permite cambiar la contraseña del usuario en la base de datos. SQL*Plus solicita la contraseña actual y la nueva sin mostrarlas en pantalla.

Sintaxis

SQL

```
PASSWORD [usuario]
```

Parámetros

- **usuario:** Nombre del usuario cuyo password se cambiará. Si se omite, se cambia el del usuario conectado.

Ejemplo

SQL

```
SQL> PASSWORD
Changing password for ALU0101618586
Password changed
```



2.3. DISCONNECT

Cierra la conexión actual con la base de datos, dejando SQL*Plus en ejecución.

Sintaxis

```
SQL  
DISCONNECT
```

Parámetros

No tiene parámetros.

Ejemplo

```
SQL  
SQL> DISCONNECT  
Disconnected from Oracle Database 12c Enterprise Edition Release 12.2.0.1.0  
- 64bit Production
```




2.4. EXIT

Cierra SQL*Plus y opcionalmente guarda o revierte los cambios pendientes en la sesión.

Sintaxis

SQL

```
EXIT [SUCCESS | FAILURE | WARNING | n | variable] [COMMIT | ROLLBACK]
```

Parámetros

- **SUCCESS**: Finaliza SQL*Plus y devuelve un código de salida 0 al sistema operativo (indica ejecución correcta).
- **FAILURE**: Devuelve un código de salida 1 (indica error).
- **WARNING**: Devuelve un código de salida 2 (advertencia).
- **n**: Devuelve el valor número que se introduzca como código de salida (*para, por ejemplo, usarlo en un script*).
- **variable**: Devuelve el valor numérico que tenga una variable como código de salida.
- **COMMIT**: Confirma (guarda) los cambios pendientes antes de salir.
- **ROLLBACK**: Deshace los cambios pendientes antes de salir.

Ejemplo

SQL

```
SQL> EXIT
```



3. Gestión buffer de instrucciones

SQL*Plus mantiene en memoria un **buffer** donde se almacena la última sentencia SQL o PL/SQL ejecutada/editada. Estos comandos permiten **visualizar, modificar, guardar o ejecutar** el contenido del buffer.



3.1. LIST

Permite visualizar líneas específicas del comando actual almacenado.

Sintaxis

SQL

```
LIST [n | n m | n LAST | * | * n | m * | * LAST | LAST]
```

Parámetros

- **n**: Muestra la línea número 'n' del buffer.
- **n m**: Muestra las líneas comprendidas entre los números 'n' y 'm' del buffer.
- **n LAST**: Muestra desde la línea 'n' hasta la última línea del buffer.
- *****: Muestra todas las líneas del buffer (en otras palabras, toda la sentencia).
- *** n**: Muestra desde la primera línea hasta la línea 'n'.
- **m ***: Muestra desde la línea 'm' hasta la última línea.
- *** LAST**: Muestra desde la primera línea hasta la última (es otra forma de mostrar todo).
- **LAST**: Muestra la última línea del buffer.



Ejemplos

SQL

```
SQL> LIST  
  1  SELECT *  
  2*  FROM DUAL
```

SQL

```
SQL> LIST 2  
  2*  FROM DUAL
```

SQL

```
SQL> LIST LAST  
  2*  FROM DUAL
```

SQL

```
SQL> LIST 1  
  1*  SELECT *
```



3.2. INPUT

Añade nuevas líneas al final del buffer actual.

Sintaxis

```
SQL  
INPUT [texto]
```

Parámetros

- **texto:** Línea (o líneas) de texto que se añadirán al buffer. Si se omite, SQL*Plus solicita las líneas hasta que se ingrese una línea vacía.

Ejemplos

```
SQL  
SQL> INPUT Nueva linea agnadida uno  
  
SQL> LIST  
1  SELECT *  
2  FROM DUAL  
3* Nueva linea agnadida uno  
  
SQL> INPUT Nueva linea agnadida dos  
  
SQL> LIST  
1  SELECT *  
2  FROM DUAL  
3  Nueva linea agnadida uno  
4* Nueva linea agnadida dos
```



3.3. APPEND

Añade texto al final de la última línea del comando almacenado.

Sintaxis

```
SQL
APPEND [texto]
```

Parámetros

- **texto:** Línea (o líneas) de texto que se añadirán a la última línea del buffer.

Ejemplo

```
SQL
SQL> APPEND Nuevo texto agnadido tras from dual
  2* FROM DUALNuevo texto agnadido tras from dual

SQL> LIST
  1 SELECT *
  2* FROM DUALNuevo texto agnadido tras from dual
```



3.4. CHANGE

Sustituye texto dentro de la línea actual del buffer. Si el texto no se encuentra, no se modifica nada.

Sintaxis

```
SQL  
CHANGE /texto_antiguo/texto_nuevo/
```

Parámetros

- **texto_antiguo:** Texto que se desea reemplazar.
- **texto_nuevo:** Texto que sustituirá al anterior.

Ejemplo

```
SQL  
SQL> CHANGE /DUAL/TABLA1/  
2* FROM TABLA1  
  
SQL> LIST  
1 SELECT *  
2* FROM TABLA1
```



3.5. DEL

Permite borrar una o varias líneas del buffer actual.

Sintaxis

SQL

```
DEL [n | n m | n LAST | * | * n | m * | * LAST | LAST]
```

Parámetros

- **n**: Borra la línea número 'n' del buffer.
- **n m**: Borra las líneas comprendidas entre los números 'n' y 'm' del buffer.
- **n LAST**: Borra desde la línea 'n' hasta la última línea del buffer.
- *****: Borra todas las líneas del buffer (en otras palabras, toda la sentencia).
- *** n**: Borra desde la primera línea hasta la línea 'n'.
- **m ***: Borra desde la línea 'm' hasta la última línea.
- *** LAST**: Borra desde la primera línea hasta la última (es otra forma de mostrar todo).
- **LAST**: Borra la última línea del buffer.



Ejemplos

SQL

```
SQL> DEL 2
```

```
SQL> LIST
```

```
1* SELECT *
```

SQL

```
SQL> LIST
```

```
1 Linea uno
```

```
2 Linea dos
```

```
3 Linea tres
```

```
4* Linea cuatro
```

```
SQL> DEL 2 3
```

```
SQL> LIST
```

```
1 Linea uno
```

```
2* Linea cuatro
```



3.6. SAVE

Guarda la sentencia actual en un archivo **'`.sql`'**.

Sintaxis

```
SQL
SAVE nombre_archivo [CREATE | REPLACE | APPEND]
```

Parámetros

- **nombre_archivo**: Nombre del archivo donde se guardará el contenido del buffer (se añade/asume la extensión **'`.sql`'** automáticamente).
- **CREATE**: Crea el archivo (da error si ya existe).
- **REPLACE**: Sobrescribe el archivo existente.
- **APPEND**: Añade el contenido del buffer al final del archivo existente.

Ejemplo

```
SQL
SQL> SAVE ContenidoBufferTest1
Created file ContenidoBufferTest1.sql
```



3.7. GET

Carga en el buffer el contenido de un archivo.

Sintaxis

```
SQL  
GET nombre_archivo [LIST | NOLIST]
```

Parámetros

- **nombre_archivo**: Nombre del archivo cuyo contenido se cargará en el buffer (se añade/asume la extensión `‘.sql’` automáticamente).
- **LIST**: Muestra el contenido del archivo cargado.
- **NOLIST**: Carga el archivo sin mostrar su contenido.

Ejemplo

```
SQL  
SQL> GET ContenidoBufferTest1  
1  SELECT *  
2*  FROM DUAL
```



3.8. RUN

Muestra la sentencia actual del buffer y la ejecuta inmediatamente.

Sintaxis

```
SQL  
RUN
```

Parámetros

No tiene parámetros.

Ejemplo

```
SQL  
SQL> RUN  
  1  SELECT *  
  2*  FROM DUAL  
  
D  
-  
X
```



3.9. /

Ejecuta la sentencia actual del buffer inmediatamente sin mostrarla (a diferencia del [RUN](#), que sí la muestra).

Sintaxis

```
SQL  
/
```

Parámetros

No tiene parámetros.

Ejemplo

```
SQL  
SQL> /  
  
D  
-  
X
```



4. Gestión de scripts

Los scripts permiten automatizar tareas mediante archivos de texto que contienen **secuencias de comandos SQL y SQL*Plus**. Estos comandos sirven para **crear, ejecutar y documentar scripts**, así como para copiar datos entre bases de datos.



4.1. EDIT

Abre el editor de texto configurado (por defecto, el del sistema operativo) para modificar un script o la instrucción contenida en el buffer.

Sintaxis

```
SQL  
EDIT [archivo]
```

Parámetros

- **nombre_archivo:** Nombre del archivo a editar. Si se omite, se edita el contenido actual del buffer en el editor predeterminado del sistema.

Ejemplos

```
SQL  
SQL> EDIT ComandoShowUserTest
```

```
SQL  
SHOW USER  
~  
~  
"ComandoShowUserTest.sql" 1L, 10C written  
1,9          All
```



SQL

SQL> EDIT

SQL

SELECT *

FROM TABLA1

/

~

~

"afiedt.buf" 3L, 23C written

2,11 All



4.2. REMARK

Permite incluir comentarios en los scripts SQL*Plus. No se ejecuta, solo sirve para documentación interna. '-- texto' y '/* texto */' son los comentarios de una sola línea y multilínea estándar respectivamente en SQL.

Sintaxis

```
SQL  
REMARK texto
```

Parámetros

- **texto:** Texto que se utilizará como comentario dentro del script.

Ejemplo

```
SQL  
SQL> REMARK Comentario que sirve para documentar  
  
SQL> REMARK -- Comentario que sirve para documenta  
  
SQL> REMARK /* Comentario que sirve para documentar */
```



4.3. START / @

Ejecuta el contenido de un archivo de script SQL. El símbolo '@' es equivalente a START, mientras que el doble símbolo '@@' se utiliza cuando un script llama a otro script que se encuentra en el mismo directorio que el archivo actualmente en ejecución.

Sintaxis

```
SQL
START nombre_archivo.sql
@nombre_archivo.sql
```

Parámetros

- **nombre_archivo.sql**: Nombre del script SQL que se ejecutará.

Ejemplos

```
SQL
SQL> START ComandoShowUserTest
USER is "ALU0101618586"

SQL> @ComandoShowUserTest
USER is "ALU0101618586"
```



5. Gestión del entorno SQL*Plus

Estos comandos controlan la **configuración del entorno**, la **visualización de resultados** y la **interacción con el sistema operativo**.



5.1. DESCRIBE

Muestra la definición de las columnas, tipos de datos y restricciones del objeto especificado.

Sintaxis

```
SQL  
DESCRIBE objeto
```

Parámetros

- **objeto:** Nombre del objeto de base de datos (como, por ejemplo, una tabla) cuya estructura se desea consultar.

Ejemplo

```
SQL  
SQL> DESCRIBE DUAL  
Name                               Null?    Type  
-----  
DUMMY                               VARCHAR2(1)
```



5.2. SPOOL

Guarda los resultados que aparecen en pantalla en un archivo. Es útil para generar informes, respaldos o mantener un historial sobre lo que se hace. Cuando se ejecuta sin parámetros, el comando muestra el estado actual del spool, indicando si hay un archivo activo o no.

Sintaxis

```
SQL  
SPOOL [nombre_archivo [APPEND] | OFF | OUT]
```

Parámetros

- **nombre_archivo:** Nombre del archivo donde se guardará toda la salida de resultados (se añade/asume la extensión `'.sql'` automáticamente).
- **APPEND:** Abre el archivo indicado en modo adjuntar, añadiendo la nueva salida al final del archivo existente sin sobrescribir su contenido.
- **OFF:** Detiene el envío de salida al archivo.
- **OUT:** Cierra el archivo actual de spool y abre uno nuevo con el mismo nombre.



Ejemplo

```
SQL
SQL> SPOOL P1 APPEND
SQL> SPOOL
currently spooling to P1.1st
SQL> SPOOL OFF
SQL> SPOOL
not spooling currently
```

5.3. SHOW

Permite visualizar valores actuales de configuraciones o variables de SQL*Plus.

Sintaxis

```
SQL
SHOW [parámetro | ALL]
```

Parámetros

- **parámetro:** Nombre del parámetro del entorno o variable que se desea visualizar (por ejemplo, LINESIZE o USER).
- **ALL:** Muestra todos los parámetros y configuraciones del entorno de SQL*Plus.



Ejemplos

SQL

```
SQL> SHOW USER
```

```
USER is "ALU0101618586"
```

SQL

```
SQL> SHOW PAGESIZE
```

```
pagesize 14
```



5.4. SET

Modifica configuraciones del entorno SQL*Plus, como el formato de salida, la paginación o el comportamiento de los comandos.

Sintaxis

```
SQL  
SET parámetro valor
```

Parámetros

- **parámetro:** Nombre del parámetro del entorno o variable que se desea modificar (por ejemplo, LINESIZE o USER).
- **valor:** Nuevo valor que se asignará al parámetro. Puede ser numérico, booleano (ON/OFF) o texto según el parámetro.

Ejemplo

```
SQL  
SQL> SET PAGESIZE 25  
  
SQL> SHOW PAGESIZE  
pagesize 25
```




5.5. HELP

Muestra información y ayuda sobre los comandos de SQL*Plus disponibles. Si no se introduce el parámetro, indica cómo acceder al índice de temas disponibles sobre los que ofrece ayuda.

Sintaxis

```
SQL  
HELP [comando]
```

Parámetros

- **comando:** Nombre del comando sobre el cual se desea obtener ayuda. Si se omite, muestra los temas disponibles.

Ejemplo

```
SQL  
SQL> HELP RUN
```

```
RUN
```

```
---
```

Lists and executes the most recently executed SQL command or PL/SQL block which is stored in the SQL buffer. The buffer has no command history list and does not record SQL*Plus commands.

```
R[UN]
```



5.6. HOST

Permite ejecutar comandos del sistema operativo sin salir de SQL*Plus.

Sintaxis

```
SQL  
HOST [comando]
```

Parámetros

- **comando**: Comando del sistema operativo que se desea ejecutar. Si se omite, SQL*Plus abre un subshell temporal.

Ejemplo

```
SQL  
SQL> HOST pwd  
/home/alumnos.ull.es/ad/alu0101618586/datos/Disco_Duro_Virtual/home
```



6. Gestión de variables

SQL*Plus permite definir **variables de sustitución y de enlace (bind variables)** para reutilizar valores en scripts, interactuar con el usuario o pasar datos entre SQL y PL/SQL.

Las **variables de sustitución** se definen y gestionan directamente por **SQL*Plus** (no por Oracle Database). Su propósito es **sustituir valores dentro de comandos o scripts antes de su ejecución**. Se identifican mediante los prefijos **'&'** o **'&&'** y permiten que el usuario introduzca o reutilice valores. Su tipo es siempre texto.

Las **bind variables** (variables de enlace) se definen con el comando **'VARIABLE'** y son gestionadas por **Oracle Database**, no por SQL*Plus. Permiten **intercambiar valores entre el entorno SQL*Plus y bloques PL/SQL**. Su valor se mantiene disponible mientras dure la sesión. Su tipo puede ser numérico, booleano, de fecha, etc.



Diferencia entre & y &&

'&' referencia una variable de sustitución. Cada vez que se utiliza, SQL*Plus solicita el valor al usuario, tal y como vemos en el ejemplo:

```
SQL
SQL> SELECT *
      2 FROM &dual;
Enter value for dual: DUAL
old   2: FROM &dual
new   2: FROM DUAL

D
-
X

SQL> DEFINE dual
SP2-0135: symbol dual is UNDEFINED
```

Por otra parte, '&&' también referencia una variable de sustitución, pero una vez solicitamos el valor al usuario, el sistema lo guarda tal y como si utilizáramos el comando DEFINE:

```
SQL
SQL> SELECT *
      2 FROM &&dual;
Enter value for dual: DUAL
old   2: FROM &&dual
new   2: FROM DUAL

D
-
X

SQL> DEFINE dual
DEFINE DUAL          = "DUAL" (CHAR)
```



6.1. DEFINE

Crea una variable de sustitución que puede ser utilizada en comandos mediante '&variable' o '&&variable'. Si ya existe la variable, muestra su contenido.

Sintaxis

```
SQL  
DEFINE variable = valor
```

Parámetros

- **variable:** Nombre de la variable de sustitución (sin el prefijo '&').
- **valor:** Valor asignado a la variable (recordemos que las variables de sustitución son siempre de tipo string).

Ejemplo

```
SQL  
SQL> DEFINE jugador = 'Goorgelas'  
SQL> DEFINE jugador  
DEFINE JUGADOR          = "Goorgelas" (CHAR)
```



6.2. UNDEFINE

Elimina una variable de sustitución definida previamente con el comando 'DEFINE'.

Sintaxis

```
SQL  
UNDEFINE variable
```

Parámetros

- **variable:** Nombre de la variable de sustitución que se quiere eliminar.

Ejemplo

```
SQL  
SQL> UNDEFINE jugador  
  
SQL> DEFINE jugador  
SP2-0135: symbol jugador is UNDEFINED
```



6.3. ACCEPT

Solicita datos al usuario (normalmente se usa durante la ejecución de un script), almacenándolos en una variable de sustitución.

Sintaxis

SQL

```
ACCEPT variable [CHAR | NUMBER | DATE] [FORMAT formato] [PROMPT texto]
```

Parámetros

- **variable:** Nombre de la variable de sustitución donde se guardará el valor introducido por el usuario.
- **CHAR:** Especifica que el valor introducido será texto (cadena de caracteres).
- **NUMBER:** Indica que el valor esperado es numérico. SQL*Plus validará que se trate de un número.
- **DATE:** Indica que el valor debe ser una fecha, con un formato compatible con Oracle (por ejemplo, DD/MM/YYYY).
- **FORMAT formato:** Define un formato específico para validar la entrada del usuario (por ejemplo, '9999' para números de 4 dígitos o 'DD/MM/YYYY' para fechas).
- **PROMPT texto:** Mensaje mostrado en pantalla para solicitar la entrada del usuario (debe ir entre comillas si incluye espacio).



Ejemplo

SQL

```
SQL> ACCEPT numero_favorito NUMBER PROMPT 'Introduzca su numero favorito: '  
Introduzca su numero favorito: 97  
SQL> DEFINE numero_favorito  
DEFINE NUMERO_FAVORITO = 97 (NUMBER)
```




6.4. VARIABLE

Declara variables (denominadas variables de enlace) que se pueden utilizar en un script o sesión. A diferencia de las variables de sustitución, las variables de enlace pueden tener un tipo de datos específico, como números o cadenas de caracteres, y pueden ser utilizadas en expresiones o asignadas a valores dentro del script.

Si se ejecuta sin parámetros, muestra todas las variables de enlace declaradas. Si se ejecuta sólo con un nombre de variable, muestra la información sobre ella (si existe).

Sintaxis

```
SQL  
VARIABLE [variable [{NUMBER | CHAR(n)} = value]
```

Parámetros

- **variable:** Nombre de la variable a declarar.
- **NUMBER:** Especifica que la variable será de tipo número.
- **CHAR(n):** Define que la variable será de tipo texto (cadena de caracteres), con una longitud máxima de 'n' caracteres. Si no se especifica 'n', por defecto la longitud será 1.
- **value:** El valor que será asignado a la variable de enlace.



Ejemplos

SQL

```
SQL> VARIABLE nombre CHAR(4) = 'hola'
```

```
SQL> VARIABLE
```

```
variable    nombre
```

```
datatype    CHAR(4)
```

```
SQL> VARIABLE nombre
```

```
variable    nombre
```

```
datatype    CHAR(4)
```



6.5. PRINT

Muestra en pantalla el valor actual de una variable de enlace creada con 'VARIABLE'. Si no se le especifican parámetros, muestra todas las variables activas.

Sintaxis

```
SQL  
PRINT [variable]
```

Parámetros

- **variable:** Nombre de la variable de enlace que se desea mostrar.

Ejemplos

```
SQL  
SQL> PRINT nombre
```

NOMBRE

Georgelas



SQL

SQL> PRINT

NOMBRE

Goorgelas

SALUDO

hola



7. Presentación de resultados

Estos comandos controlan **la forma en que se muestran los resultados** en pantalla o en archivos, permitiendo añadir títulos, pausas, encabezados y formato de columnas.



7.1. PAUSE

Detiene la ejecución de un script hasta que el usuario presione 'ENTER'. Es útil para informes interactivos. Si no se introduce ningún parámetro, no muestra ningún texto.

Sintaxis

```
SQL  
PAUSE [texto]
```

Parámetros

- **texto:** Mensaje mostrado antes de pausar la ejecución.

Ejemplos

```
SQL  
SQL> PAUSE  
  
SQL> PAUSE Esto es una pausa  
Esto es una pausa
```



7.2. PROMPT

Muestra un mensaje informativo en pantalla durante la ejecución del script.

Sintaxis

```
SQL  
PROMPT [texto]
```

Parámetros

- **texto:** Texto que se mostrará en pantalla.

Ejemplo

```
SQL  
SQL> PROMPT Esto es un mensaje informativo  
Esto es un mensaje informativo
```



7.3. TTITLE

Muestra un título superior en los resultados de las consultas. Este título aparece al inicio de cada página de resultados y puede ser personalizado con texto estático o dinámico, así como formateado con diferentes opciones como alineación y saltos de línea.

Sintaxis

SQL

```
TTITLE [opción [texto | variable] | OFF | ON ]
```

Parámetros

- **opción:** Instrucciones opcionales de formato adicional que se pueden agregar al título, como la alineación o el salto de línea (como, por ejemplo, 'SKIP' o 'CENTER').
- **texto:** El texto fijo que se mostrará como título en los resultados. También puede incluir texto concatenado con variables.
- **variable:** El valor de una variable (por ejemplo, '&usuario') que se usará dinámicamente como título.
- **ON:** Activa el título superior (aparece en la parte superior de cada página de salida) en los resultados.
- **OFF:** Desactiva el título superior (lo elimina de todas las páginas de salida).



Ejemplo

SQL

```
SQL> TTITLE 'TITULO SUPERIOR'
```

SQL

```
SQL> TTITLE 'TITULO SUPERIOR PATROCINADO POR &&jugador'
```



7.4. BTITLE

Muestra un pie de página al final de cada página de resultados en SQL*Plus. Este pie de página puede ser personalizado con texto fijo o dinámico y formateado con opciones adicionales como alineación y saltos de línea.

Sintaxis

```
SQL  
BTITLE [opción [texto | variable] | OFF | ON ]
```

Parámetros

- **opción:** Instrucciones opcionales de formato adicional que se pueden agregar al pie de página, como la alineación o el salto de línea (como, por ejemplo, 'SKIP' o 'CENTER').
- **texto:** El texto fijo que se mostrará como pie de página en los resultados. También puede incluir texto concatenado con variables.
- **variable:** El valor de una variable (por ejemplo, '&usuario') que se usará dinámicamente en el pie de página.
- **ON:** Activa el título inferior (aparece en el pie de página de cada página de salida) en los resultados.
- **OFF:** Desactiva el título inferior (lo elimina de todas las páginas de salida).



Ejemplo

```
SQL
SQL> 'BTITLE PIE DE PAGINA'
```

```
SQL
SQL> BTITLE 'PIE DE PAGINA PATROCINADO POR &&jugador'
```

Resultados de TTITLE y BTITLE

```
SQL
SQL> SELECT *
      2 FROM DUAL;

Sun Nov 16                                     page
1
          TITULO SUPERIOR PATROCINADO POR Goorgelas

D
-
X

          PIE DE PAGINA PATROCINADO POR Goorgelas
```



7.5. COLUMN

Personaliza cómo se muestran las columnas (o expresiones) en los resultados de una consulta. Es muy útil para dar formato a las columnas en informes, como alinear texto o cambiar el encabezado.

Sintaxis

```
SQL  
COLUMN {columna | expresión} opción
```

Parámetros

- **columna:** Nombre de la columna en la que se desea aplicar el formato.
- **expresión:** Expresión que puede ser el resultado de una función o cálculo (no necesariamente un nombre de columna).
- **opción:** Opciones que definen cómo se formatea la columna o expresión, tales como el encabezado, el formato y la alineación (como, por ejemplo, 'HEADING' o 'FORMAT').

Ejemplo

```
SQL  
COLUMN dni HEADING 'DNI' FORMAT A12
```



8. Referencias

[1] **Oracle Corporation** (2017). *SQL*Plus® Quick Reference. Release 2 (12.2)*. E50027-07. Disponible en: <https://www.oracle.com> [Accedido: 26 octubre 2025].

[2] **Guion de la práctica 6 de la asignatura BBDD** (curso 2025/2026). Disponible en: <https://campusvirtual.ull.es/2526/ingenieriauytecnologia/mod/resource/view.php?id=18890> [Accedido: 7 diciembre 2025].

[3] Ruiz Ruiz, Francisco Gabriel (2025). **Spools e informes de la asignatura Bases de Datos** (ULL, curso 2025/2026) [Repositorio *GitHub*]. Disponible en: <https://github.com/Francisco-Gabriel-Ruiz-Ruiz/Spools-AsignaturaBBD-D-ULL-2526.git> [Accedido: 7 diciembre 2025].